Sistema de Pedidos de Medicamentos — Farmacia

> Laboratorio #2 · Diseño e Implementación de Interfaz de Usuario
> Java Swing · Visual Studio Code · Clean Code · Programador Pragmático

> Participantes Deisy Jimena Simanca Castro, Victoria Pilar Carvajal, Brayan Felipe Tovar Hernandez.

¿Qué hace este proyecto?

Este proyecto simula el sistema de pedidos de medicamentos de una farmacia hacia un distribuidor farmacéutico. El usuario completa un formulario con los datos del pedido (medicamento, tipo, cantidad, distribuidor y sucursal de destino), el sistema valida toda la información y, si es correcta, muestra una segunda ventana con el resumen del pedido para su confirmación final y envío.

La aplicación está construida completamente en **Java con la librería Swing**, sin dependencias externas, y está diseñada para ejecutarse directamente desde **Visual Studio Code**.

Funcionalidad paso a paso

1. Inicio de la aplicación

Al ejecutar `Main.java`, se lanza la ventana principal `MainWindow` que contiene el formulario de pedido. La GUI se inicializa dentro del Event Dispatch Thread de Swing usando `SwingUtilities.invokeLater`, tal como lo requieren las buenas prácticas de Swing para evitar problemas de concurrencia en la interfaz.

2. Formulario de pedido — Ventana 1

El usuario ve el panel `FormularioPedidoPanel` con los siguientes campos:

Campo	Componente Swing	Descripción
Nombre del medicamento	`JTextField`	Texto libre alfanumérico
Tipo de medicamento	`JComboBox`	6 opciones del enum `TipoMedicamento`
Cantidad requerida	`JTextField`	Número entero positivo
Distribuidor	`JRadioButton`	Uno de los 3 del enum `Distribuidor`
Sucursal de destino	`JCheckBox`	Una o ambas del enum `Sucursal`

Al finalizar, el usuario puede:

- Pulsar **Borrar**: limpia todos los campos y devuelve el foco al primer campo.
- Pulsar **Confirmar**: desencadena el proceso de validación.

3. Validación de datos

Cuando el usuario pulsa Confirmar, `FormularioPedidoPanel` delega la verificación al `PedidoValidator`, que comprueba 5 reglas de negocio:

1. El nombre del medicamento no está vacío y contiene caracteres alfanuméricos.
2. Se ha seleccionado un tipo de medicamento del combo.
3. La cantidad es un número entero mayor que cero.
4. Se ha elegido exactamente un distribuidor.
5. Se ha marcado al menos una sucursal de destino.

El resultado se encapsula en un objeto `ValidationResult`. Si hay errores, se muestran todos de una vez en un `JOptionPane`. Si todo es correcto, se construye el objeto `Pedido` y se abre la segunda ventana.

4. Resumen del pedido — Ventana 2

Se abre `ResumenPedidoDialog`, un `JDialog` modal, con el título *"Pedido al distribuidor [nombre]"* y la siguiente información:

- Descripción: *"X unidades del [tipo] [nombre]"*
- Distribuidor seleccionado.
- Dirección(es) de la(s) sucursal(es), tomadas del enum `Sucursal`.

El usuario puede:

- Pulsar **Cancelar**: cierra el diálogo sin enviar nada, volviendo al formulario.
- Pulsar **Enviar Pedido**: llama a `PedidoService.enviarPedido()`, imprime el resumen por consola y muestra un mensaje de confirmación.

Arquitectura del proyecto

El proyecto sigue una **arquitectura en capas** donde cada paquete tiene una responsabilidad bien delimitada:

```
com.farmacia/
│
├── Main.java                        ← Entry point
│
├── model/                           ← Capa de dominio
│   ├── Pedido.java
│   ├── TipoMedicamento.java
│   ├── Distribuidor.java
│   └── Sucursal.java
│
├── service/                         ← Capa de negocio
│   └── PedidoService.java
│
├── validator/                       ← Capa de validación
│   ├── PedidoValidator.java
│   └── ValidationResult.java
│
├── ui/                              ← Capa de presentación
```

```
│   ├── MainWindow.java
│   ├── FormularioPedidoPanel.java
│   └── ResumenPedidoDialog.java
│
└── util/                          ← Capa transversal
    └── UIUtils.java
```

Descripción de cada clase

`Main.java`

Punto de entrada. Configura el LookAndFeel del sistema con `UIUtils.configurarLookAndFeel()` y lanza `MainWindow` dentro del Event Dispatch Thread con `SwingUtilities.invokeLater`.

`model/Pedido.java`

Entidad principal del dominio. Clase **inmutable**: todos sus campos son `final` y la lista de sucursales se copia defensivamente con `List.copyOf()`. Incluye `getDescripcionMedicamento()` que genera el texto *"X unidades del [tipo] [nombre]"* mostrado en el resumen.

`model/TipoMedicamento.java`

Enum con 6 tipos: Analgésico, Analéptico, Anestésico, Antiácido, Antidepresivo y Antibiótico. Cada valor lleva su nombre de visualización para el `JComboBox` y el resumen.

`model/Distribuidor.java`

Enum con 3 distribuidores: Cofarma, Empsephar y Cemefar. Encapsula el nombre oficial para los `JRadioButton` y el título de la ventana de resumen.

`model/Sucursal.java`

Enum con 2 sucursales: Principal y Secundaria. Cada valor lleva su nombre y su **dirección física** (Calle de la Rosa n. 28 y Calle Alcazabilla n. 3). La dirección siempre va ligada a la sucursal, sin necesidad de buscarla en otro lugar.

`service/PedidoService.java`

Contiene la lógica de negocio del envío. El método `enviarPedido(Pedido pedido)` recibe el pedido ya validado y lo procesa. Actualmente simula el envío por consola. En producción, aquí iría la integración con una API o base de datos sin tocar ninguna otra clase.

`validator/PedidoValidator.java`

Responsabilidad única: verificar los datos del formulario. Aplica las 5 reglas de forma independiente, acumula todos los errores en una lista y devuelve siempre un `ValidationResult`. Nunca lanza

excepciones.

`validator/ValidationResult.java`

Implementa el **patrón Result Object**. Encapsula una lista inmutable de mensajes de error. Métodos clave: `esValido()` y `getMensajeErrores()`. Se construye con los métodos de fábrica `ValidationResult.exitoso()` y `ValidationResult.conErrores(lista)`.

`ui/MainWindow.java`

El `JFrame` principal. Configura la ventana e instancia `PedidoValidator` y `PedidoService` para inyectarlos en `FormularioPedidoPanel`. La capa de presentación no crea sus propias dependencias.

`ui/FormularioPedidoPanel.java`

Panel central de la aplicación. Gestiona todos los controles Swing y coordina el flujo:

1. Recoge los valores de todos los campos.
2. Los pasa al `PedidoValidator`.
3. Si hay errores, los muestra con `UIUtils.mostrarErrores()`.
4. Si son correctos, construye el `Pedido` y abre `ResumenPedidoDialog`.

El método `limpiarFormulario()` resetea todos los componentes al pulsar Borrar.

`ui/ResumenPedidoDialog.java`

`JDialog` modal de confirmación. Recibe el `Pedido` ya construido y lo muestra de forma legible. Al pulsar Enviar, delega en `PedidoService` y cierra el diálogo. Al pulsar Cancelar, solo cierra sin enviar, dejando el formulario intacto.

`util/UIUtils.java`

Clase de utilidad estática no instanciable. Centraliza toda la configuración visual: paleta de colores, fuentes y métodos fábrica (`crearBotonPrimario`, `crearBotonSecundario`, `crearCampoTexto`, `crearBordeSección`). Principio **DRY**: un único lugar para cambiar cualquier aspecto visual de toda la aplicación.

- **Botón Confirmar / Enviar**: fondo azul cielo `#4FC3F7`, borde azul oscuro, texto negro.
- **Botón Borrar / Cancelar**: fondo ámbar `#FFCC80`, borde naranja oscuro, texto negro.

Integración entre clases

```
Main
 └─▶ MainWindow
        │ instancia e inyecta
        ├─▶ PedidoValidator
        ├─▶ PedidoService
        └─▶ FormularioPedidoPanel
```

```
                    |
            [pulsa Confirmar]
                    |
                    ▼
           PedidoValidator.validar()
                    |
          ┌─────────┴──────────┐
       inválido              válido
          |                    |
          ▼                    ▼
    mostrarErrores()        new Pedido()
    (JOptionPane)                |
                                 ▼
                      ResumenPedidoDialog
                                 |
                       [pulsa Enviar]
                                 |
                                 ▼
                    PedidoService.enviarPedido()
                     → imprime por consola
                     → muestra confirmación
```

Principios de diseño aplicados

Clean Code

- **Nombres expresivos**: cada clase y método describe su propósito sin comentarios adicionales.
- **Responsabilidad única (SRP)**: `PedidoValidator` solo valida, `PedidoService` solo envía, `UIUtils` solo gestiona estilos visuales.
- **Métodos cortos**: los métodos largos están descompuestos en métodos privados con nombre descriptivo.
- **Sin magic strings ni magic numbers**: todos los valores fijos están en enums o constantes con nombre.

Programador Pragmático

- **Modelo inmutable**: `Pedido` no puede modificarse tras su construcción.
- **Result Object**: `ValidationResult` evita usar excepciones para controlar flujo normal.
- **Enums con comportamiento**: `Sucursal` lleva su dirección, `TipoMedicamento` lleva su nombre de visualización.
- **DRY**: `UIUtils` es la única fuente de verdad para los estilos.
- **Separación estricta de capas**: la UI no contiene lógica de negocio; el servicio no conoce Swing.

Cómo ejecutar en VS Code

Requisitos:

- Java JDK 11 o superior

- VS Code con la extensión **Extension Pack for Java**

Pasos:

1. Abrir la carpeta `farmacia/` en VS Code.
2. Esperar a que la extensión Java indexe el proyecto.
3. Abrir `Main.java` y pulsar ▷ **Run Java**.

Por terminal:

```
mkdir -p bin
find src -name "*.java" | xargs javac -encoding UTF-8 -d bin
java -cp bin com.farmacia.Main
```

Resultado final esperado

Al ejecutar la aplicación el usuario verá:

1. **Ventana 1**: formulario con todos los campos, botón **Borrar** (ámbar) y **Confirmar** (azul brillante con borde).
2. Si los datos son incorrectos: diálogo de error listando todos los problemas encontrados.
3. Si los datos son correctos: **Ventana 2** modal con el resumen del pedido y botones **Cancelar** y **Enviar Pedido**.
4. Al enviar: resumen impreso en consola y mensaje de confirmación en pantalla.

Uso de AWT y Swing en el proyecto

El enunciado del laboratorio indica trabajar con **AWT/Swing**. En Java, Swing está construido encima de AWT, por lo que ambas tecnologías coexisten y se complementan en este proyecto.

¿Cómo se reparten las responsabilidades?

AWT (`java.awt`) pone los cimientos — gestiona layouts, colores, fuentes y eventos del sistema operativo. **Swing (`javax.swing`) construye los componentes visuales encima** de esa base.

Lo que aporta AWT en este proyecto

Clase AWT	Dónde se usa	Para qué
`Color`	`UIUtils.java`	Define toda la paleta de colores de la aplicación
`Font`	`UIUtils.java`	Fuentes de etiquetas, campos y botones
`Dimension`	`UIUtils.java`	Tamaño preferido de los botones

Clase AWT	Dónde se usa	Para qué
`Cursor`	`UIUtils.java`	Cursor de mano al pasar sobre los botones
`Component`	`UIUtils.java`	Parámetro base en `mostrarErrores()`
`BorderLayout`	`FormularioPedidoPanel`, `ResumenPedidoDialog`	Divide los paneles en zonas (NORTH, CENTER, SOUTH)
`GridBagLayout`	`FormularioPedidoPanel`	Alinea etiquetas y campos del formulario en rejilla
`GridBagConstraints`	`FormularioPedidoPanel`	Controla expansión y posición de cada celda
`FlowLayout`	Varios paneles	Alinea botones y grupos de controles en fila
`Insets`	`FormularioPedidoPanel`	Espaciado interno entre celdas del grid

Lo que aporta Swing en este proyecto

Clase Swing	Dónde se usa	Para qué
`JFrame`	`MainWindow`	Ventana principal de la aplicación
`JPanel`	`FormularioPedidoPanel`	Panel contenedor del formulario
`JDialog`	`ResumenPedidoDialog`	Ventana modal de resumen del pedido
`JTextField`	Formulario	Campos de nombre y cantidad
`JComboBox`	Formulario	Desplegable de tipos de medicamento
`JRadioButton`	Formulario	Selección exclusiva de distribuidor
`JCheckBox`	Formulario	Selección de sucursal(es) de destino
`JButton`	Formulario y resumen	Botones Confirmar, Borrar, Enviar, Cancelar
`JLabel`	`ResumenPedidoDialog`	Textos del resumen del pedido
`JOptionPane`	`UIUtils`, `ResumenPedidoDialog`	Diálogos de error y confirmación
`ButtonGroup`	Formulario	Agrupa los `JRadioButton` para exclusividad
`BorderFactory`	`UIUtils`	Crea bordes compuestos para campos y secciones
`SwingUtilities`	`Main`	Lanza la GUI en el Event Dispatch Thread

Conclusión

Aunque visualmente todo lo que el usuario ve son componentes Swing, por debajo AWT está presente en cada layout, cada color y cada fuente del proyecto. La separación es clara: AWT gestiona la infraestructura gráfica del sistema operativo y Swing proporciona los controles de alto nivel que el usuario interactúa directamente.